



UNIVERSIDADE DO MINHO

BASES DE DADOS CLÍNICAS E DE GESTÃO HOSPITALAR

Sistema de Gestão e Manutenção de Equipamentos Médicos Hospitalares

Realizado por:

A107272 Beatriz Maria Ferreira de Freitas Ribeiro

A107195 Clara Sofia Salazar Carvalho

A107159 Dinis Soveral Botelho Rosa

A107246 Renata Madalena Ferreira Bravo

Docentes:

Hugo Peixoto

José Machado

Lídia Sousa

Braga, 22 de Maio de 2026

Índice

1	Introdução	1
1.1	Contextualização	1
1.2	Apresentação do Caso de Estudo	1
1.3	Motivação e Objetivos	2
2	Modelo Conceptual	2
2.1	Entidades	3
2.2	Atributos	4
2.3	Estudo de Viabilidade	6
2.4	Levantamento de Requisitos	6
3	Modelo Lógico	7
4	Modelo Físico	8
5	Documentação de Atributos	9
6	Povoamento	10
7	Funções	10
7.1	Função <i>calcular_idade_equipamento</i>	10
7.2	Função <i>calcular_duracao_manutencao</i>	11
7.3	Função <i>calcular_experiencia_tecnico</i>	11
7.4	Função <i>equipamento_tem_manutencao_ativa</i>	11
7.5	Função <i>obter_localizacao_equipamento</i>	11
7.6	Função <i>custo_total_manutencoes</i>	12
8	Procedures	12
8.1	Procedimento <i>registar_avaria</i>	12
8.2	Procedimento <i>concluir_manutencao</i>	12
8.3	Procedimento <i>adicionar_intervencao_tecnico</i>	12
8.4	Procedimento <i>abater_equipamento</i>	13
8.5	Procedimento <i>alterar_prioridade_ordem</i>	13
8.6	Procedimento <i>listar_alertas</i>	13
8.7	Procedimento <i>validar_estado_ordem</i>	13
8.8	Procedimento <i>adicionar_responsavel</i>	13
8.9	Procedimento <i>adicionar_tecnico</i>	13

8.10	Procedimento <i>adicionar_peca</i>	14
8.11	Procedimento <i>abater_pecas</i>	14
9	Triggers	14
9.1	Trigger <i>trg_manutencao_validar_datas_insert</i>	14
9.2	Trigger <i>trg_manutencao_validar_datas_update</i>	14
9.3	Trigger <i>trg_manutencao_estado Equipamento_insert</i>	15
9.4	Trigger <i>trg_manutencao_estado Equipamento_update</i>	15
9.5	Trigger <i>trg_ordem_servico_validar_dominios_insert</i> e <i>trg_ordem_servico_validar_dominios_update</i>	15
9.6	Trigger <i>trg Equipamento_proteger_delete</i> e <i>trg Equipamento_validar_estado_update</i>	15
9.7	Trigger <i>trg Equipamento_proteger_delete</i>	15
9.8	Trigger <i>trg_manutencao_validar_custo_insert</i>	16
10	Queries	16
10.1	Query 1 - Técnicos Disponíveis por Especialidade	16
10.2	Query 2 - Peças com Garantia a Expirar	16
10.3	Query 3 - Equipamentos sem Histórico de Manutenção	17
10.4	Query 4 - <i>Ranking</i> de Técnicos por Custo Gerido	17
10.5	Query 5 - Histórico Completo de um Equipamento	17
10.6	Query 6 - Estado Operacional dos Equipamentos por Departamento	17
10.7	Query 7 - Manutenções com Maior Custo Total	18
11	Views	18
11.1	View <i>vw_custo_total Equipamento</i>	18
11.2	View <i>vw_garantia_pecas</i>	18
11.3	View <i>vw_rastreabilidade_intervencoes</i>	19
11.4	View <i>vw_downtime Equipamentos</i>	19
11.5	View <i>vw_localizacao Equipamentos</i>	19
11.6	View <i>vw_ordens_abertas</i>	19
11.7	View <i>vw_tecnico_resumo</i>	20
11.8	View <i>vw_custos_por_departamento</i>	20
12	Conclusão	20
A	Anexo 1: Requisitos da base de dados	22
B	Anexo 2: Tabela de Atributos	24

1 Introdução

1.1 Contextualização

Na sua definição mais básica, uma base de dados é um conjunto de informações interrelacionadas, utilizadas para armazenar e organizar dados de forma a facilitar a sua gestão e o acesso. Os dados referem-se a qualquer informação capturada sobre uma entidade e os seus atributos. À medida que uma coleção de dados cresce e assume maior complexidade, torna-se mais difícil manter a organização e a acessibilidade, surgindo a base de dados como a solução central para facilitar a interpretação desta informação.

Neste âmbito, os Sistemas de Base de Dados permitem-nos armazenar, recuperar e relacionar dados de forma estruturada. A plataforma MySQL, um dos sistemas mais utilizados, serve como interface de gestão para bases de dados relacionais através da linguagem SQL. No setor da saúde, estas ferramentas são fulcrais para garantir o funcionamento eficiente das instituições, nomeadamente na gestão de dispositivos médicos. A monitorização e manutenção destes equipamentos, apoiadas por bases de dados, são essenciais para o controlo operacional diário, garantindo que a tecnologia médica é utilizada com a máxima eficiência.

A manutenção regular é fundamental para assegurar a segurança dos pacientes e a qualidade dos serviços de saúde. Equipamentos em condições subótimas podem levar a diagnósticos incorretos ou tratamentos inadequados, colocando vidas em risco. Dada a complexidade dos aparelhos hospitalares, é imprescindível um sistema organizado que facilite o agendamento de manutenções preventivas e corretivas. Um sistema bem estruturado permite antecipar falhas, minimizar interrupções e garantir que os aparelhos cumpram as normas de segurança. Além disso, a rastreabilidade dos históricos permite identificar tendências de desgaste, otimizando recursos e reduzindo custos com reparos emergenciais ou substituições prematuras, promovendo, em última análise, a sustentabilidade operacional e financeira do hospital.

1.2 Apresentação do Caso de Estudo

O presente caso de estudo foca-se no desenvolvimento de um sistema para a gestão e acompanhamento da manutenção de equipamentos hospitalares. O sistema centraliza informações sobre os dispositivos, a sua localização geográfica nas unidades de saúde e os departamentos responsáveis.

A solução proposta integra os seguintes tópicos:

- Gestão de Intervenções: Registo de ordens de serviço, históricos de manutenção e acompanhamento técnico.
- Gestão de Recursos: Monitorização dos técnicos envolvidos e inventário de peças utilizadas.
- Controlo Financeiro: Rastreabilidade dos custos associados a cada intervenção e análise de desgaste dos ativos ao longo do tempo.

1.3 Motivação e Objetivos

A motivação deste trabalho reside na aplicação prática dos conceitos de planeamento e desenvolvimento de bases de dados relacionais a um cenário real e complexo. O objetivo central é projetar uma solução que garanta a integridade e a acessibilidade dos dados hospitalares.

Para alcançar este propósito, o projeto divide-se nos seguintes objetivos específicos:

- Modelação: Elaboração dos modelos conceptual (Diagrama Entidade-Associação), lógico e físico.
- Implementação: Criação da base de dados em MySQL e respetivo povoamento com dados realistas.
- Programação SQL: Desenvolvimento de subprogramas avançados, incluindo procedimentos (*procedures*), funções e triggers, para automatizar tarefas e facilitar a extração de métricas relevantes para a gestão hospitalar.

2 Modelo Conceptual

Um modelo conceptual consiste numa representação gráfica que permite visualizar as ligações entre diferentes conceitos e ideias. Este tipo de modelo é desenvolvido a partir da análise de uma base de dados, na qual a informação é organizada em entidades, atributos e chaves primárias, sendo também definidos os respetivos relacionamentos entre esses elementos. Numa fase inicial, é fundamental identificar as entidades a analisar, bem como os seus atributos, estabelecendo também as relações entre elas, de modo a representar de forma coerente a lógica do mundo real.

Na notação de Chen, as entidades são representadas por retângulos verdes, os atributos por elipses de cor roxa e as relações entre entidades por losangos amarelos inseridos em retângulos verdes. Os atributos-chave destacam-se por estarem sublinhados e a negrito.

Para definir corretamente as relações, a cardinalidade assume um papel essencial, pois indica quantas ocorrências de uma entidade podem estar associadas a outra. As relações podem ser classificadas em três tipos: um para um (1:1), um para muitos (1:N) e muitos para muitos (N:N). Este último ocorre quando uma entidade pode associar-se a várias ocorrências de outra entidade e vice-versa, originando normalmente uma entidade associativa.

Relativamente às ligações entre entidades, uma linha simples indica que a participação na relação é opcional, enquanto uma linha dupla representa uma participação obrigatória. A fig. 1 apresenta o nosso diagrama Entidade-Relacionamento (ER), criado com recurso ao *software* TerraER.

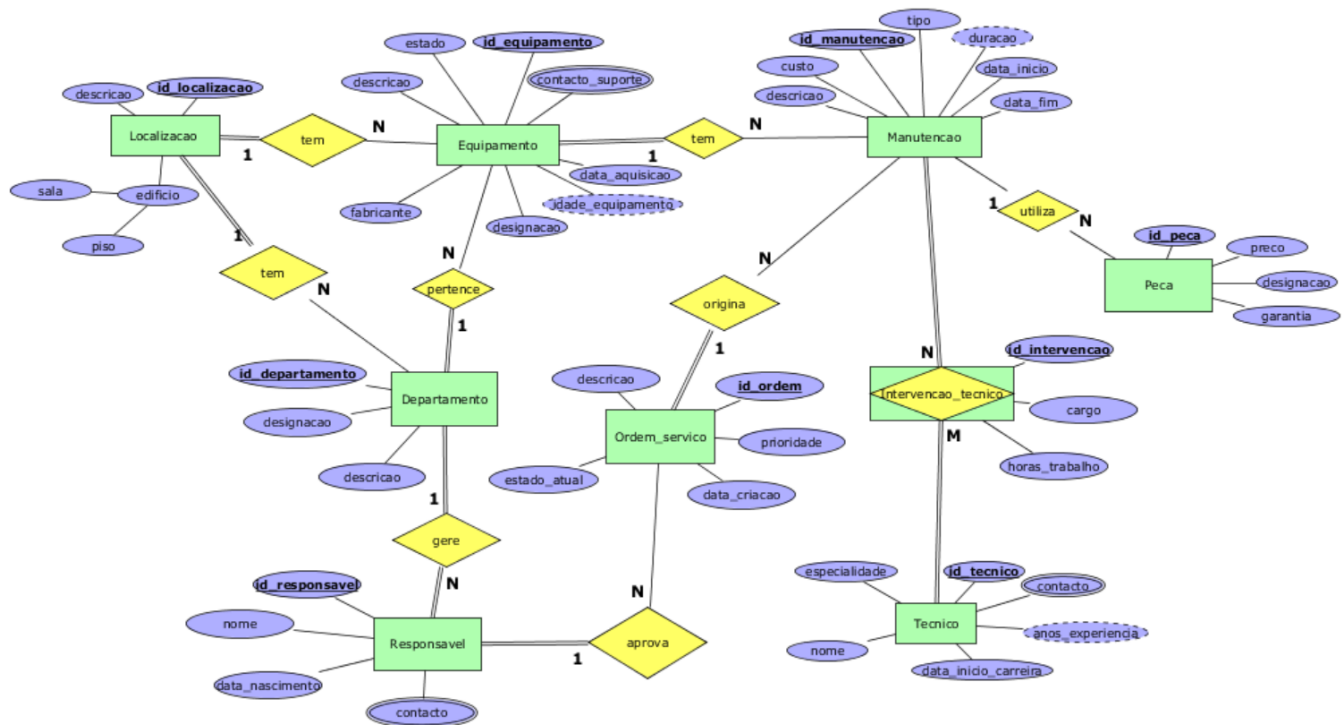


Figura 1: Modelo Conceptual.

2.1 Entidades

Entidades são objetos ou conceitos representados no banco de dados. Neste modelo, definiram-se as seguintes entidades:

- **Localizacao:** Armazena informações sobre os locais onde os equipamentos se encontram;
- **Equipamento:** Contém os dados relativos aos equipamentos existentes;
- **Manutencao:** Regista informações sobre as manutenções realizadas nos equipamentos;
- **Peca:** Armazena dados sobre as peças utilizadas nas manutenções;
- **Tecnico:** Regista os dados dos técnicos responsáveis pelas intervenções;
- **Intervencao_tecnico:** Representa a participação dos técnicos nas intervenções;
- **Ordem_servico:** Guarda informações sobre as ordens de serviço criadas;
- **Departamento:** Contém os dados dos departamentos da organização;
- **Responsavel:** Armazena informações sobre os responsáveis associados aos departamentos.

2.2 Atributos

Atributos são características ou propriedades que descrevem uma entidade. Cada atributo tem um tipo de dado associado, que define o tipo de valor que o atributo pode armazenar, como números inteiros, texto, datas, entre outros.

De seguida, apresentam-se os atributos associados a cada uma das entidades:

Localizacao:

- id_localizacao - Identificador único da localização;
- descricao - Descrição da localização;
- sala - Sala onde se encontra o equipamento;
- piso - Piso da localização;
- edificio - Edifício correspondente.

Equipamento:

- id_equipamento - Identificador único do equipamento;
- descricao - Descrição do equipamento;
- estado - Estado atual do equipamento;
- fabricante - Fabricante do equipamento;
- designacao - Nome/designação do equipamento;
- data_aquisicao - Data de aquisição do equipamento;
- idade_equipamento - Idade do equipamento;
- contacto_suporte - Contacto de suporte técnico.

Manutencao:

- id_manutencao - Identificador único da manutenção;
- tipo - Tipo de manutenção;
- duracao - Duração da manutenção;
- data_inicio - Data de início da manutenção;
- data_fim - Data de fim da manutenção;

- custo - Custo da manutenção;
- descricao - Descrição da manutenção.

Peca:

- id_peca - Identificador único da peça;
- designacao - Nome/designação da peça;
- preco - Preço da peça;
- garantia - Validade da peça.

Tecnico:

- id_tecnico - Identificador único do técnico;
- nome - Nome do técnico;
- especialidade - Especialidade do técnico;
- contacto - Contacto do técnico;
- anos_experiencia - Anos de experiência;
- data_inicio_carreira - Data de início de carreira.

Intervencao_tecnico:

- id_intervencao - Identificador único da intervenção;
- cargo - Função do técnico na intervenção;
- horas_trabalho - Número de horas trabalhadas.

Ordem_servico:

- id_ordem - Identificador único da ordem de serviço;
- descricao - Descrição da ordem;
- estado_atual - Estado atual da ordem;
- data_criacao - Data de criação da ordem;
- prioridade - Prioridade da ordem.

Departamento:

- id_departamento - Identificador único do departamento;
- designacao - Nome/designação do departamento;
- descricao - Descrição do departamento.

Responsavel:

- id_responsavel - Identificador único do responsável;
- nome - Nome do responsável;
- data_nascimento - Data de nascimento;
- contacto - Contacto do responsável.

2.3 Estudo de Viabilidade

O sistema desenvolvido revela-se viável nas vertentes técnica, operacional e legal, considerando o contexto de gestão de equipamentos distribuídos por vários departamentos e localizações físicas dentro de uma organização.

A nível técnico, a solução assenta num modelo relacional devidamente estruturado, compatível com sistemas de gestão de bases de dados amplamente utilizados no mercado. A definição de entidades permite organizar e relacionar a informação de forma eficiente. Adicionalmente, a utilização de tipos de dados standard, nomeadamente `INT`, `VARCHAR`, `DATE` e `DECIMAL`, contribui para simplificar a implementação do sistema, facilitar a sua manutenção e garantir maior capacidade de escalabilidade futura.

No plano operacional, o sistema proporciona uma centralização eficaz da informação relativa aos equipamentos e às respetivas atividades de manutenção, reduzindo redundâncias e diminuindo a probabilidade de ocorrência de erros humanos. Para além disso, possibilita o acompanhamento detalhado do ciclo de vida dos equipamentos, incluindo a sua localização física, estado atual, histórico de intervenções e técnicos envolvidos, promovendo uma gestão patrimonial mais rigorosa e eficiente.

Relativamente aos aspetos legais e éticos, o sistema deverá assegurar conformidade com o Regulamento Geral sobre a Proteção de Dados (RGPD), dado que envolve o tratamento de dados pessoais associados a responsáveis e técnicos.

2.4 Levantamento de Requisitos

Os requisitos definidos para o sistema dividem-se em requisitos funcionais, não funcionais e requisitos associados à estrutura e controlo da base de dados, detalhados no Anexo 1.

No âmbito funcional, o sistema deverá permitir o registo e gestão de equipamentos, garantindo a associação de cada equipamento ao respetivo departamento e localização física. Deverá igualmente suportar a criação, aprovação e acompanhamento de ordens de serviço. Além disso, o sistema deverá possibilitar o registo de manutenções, intervenções técnicas e peças utilizadas, permitindo consultar todo o histórico associado a cada equipamento, incluindo

intervenções realizadas e técnicos envolvidos. A solução deverá também incorporar mecanismos automáticos de gestão operacional. Sempre que uma manutenção seja iniciada ou concluída, o estado do equipamento deverá ser atualizado automaticamente entre os estados "Em Manutenção" e "Operacional", reduzindo a necessidade de intervenção manual e minimizando potenciais erros. Da mesma forma, após a conclusão de uma manutenção, a respetiva ordem de serviço deverá ser encerrada automaticamente pelo sistema.

Ao nível das regras de negócio, o sistema deverá assegurar a coerência temporal das manutenções, validando que a data de fim seja sempre posterior à data de início. Adicionalmente, deverá impedir a existência de múltiplas manutenções em simultâneo para o mesmo equipamento quando exista uma intervenção ainda em curso. Em termos financeiros, o custo total registado numa manutenção não poderá ser inferior ao valor das peças utilizadas nessa intervenção, garantindo consistência e credibilidade nos registos.

Relativamente aos requisitos não funcionais, destacam-se os aspetos relacionados com segurança, privacidade, integridade e fiabilidade da informação. A integridade dos dados deverá ser assegurada através da utilização de chaves primárias e estrangeiras, restrições de nulidade e regras de integridade referencial. A base de dados deverá assegurar um elevado nível de rastreabilidade da informação, permitindo acompanhar todo o ciclo de vida dos equipamentos, com desempenho das manutenções otimizado através de indexação apropriada e de uma estrutura eficiente de armazenamento e relacionamento dos dados.

3 Modelo Lógico

O modelo lógico de uma base de dados representa a fase de transição entre a abstração do modelo conceptual e a implementação técnica no Sistema de Gestão de Base de Dados (SGBD). Nesta etapa, os conceitos e ideias definidos anteriormente são convertidos num modelo representacional, onde as entidades dão lugar a tabelas e os atributos tornam-se colunas com tipos de dados específicos.

Ao contrário do modelo conceptual, o modelo lógico foca-se na eficiência da estrutura de dados e nas relações formais. Através de processos de normalização, procura-se obter um conjunto de tabelas que minimizem a redundância e assegurem a integridade referencial, permitindo um registo de dados robusto e coerente com a lógica do mundo real.

Com base no modelo conceptual anterior, foi elaborado um modelo lógico em MySQL, apresentado na fig. 2. Este modelo vai servir também ponto de partida para a criação do modelo físico.

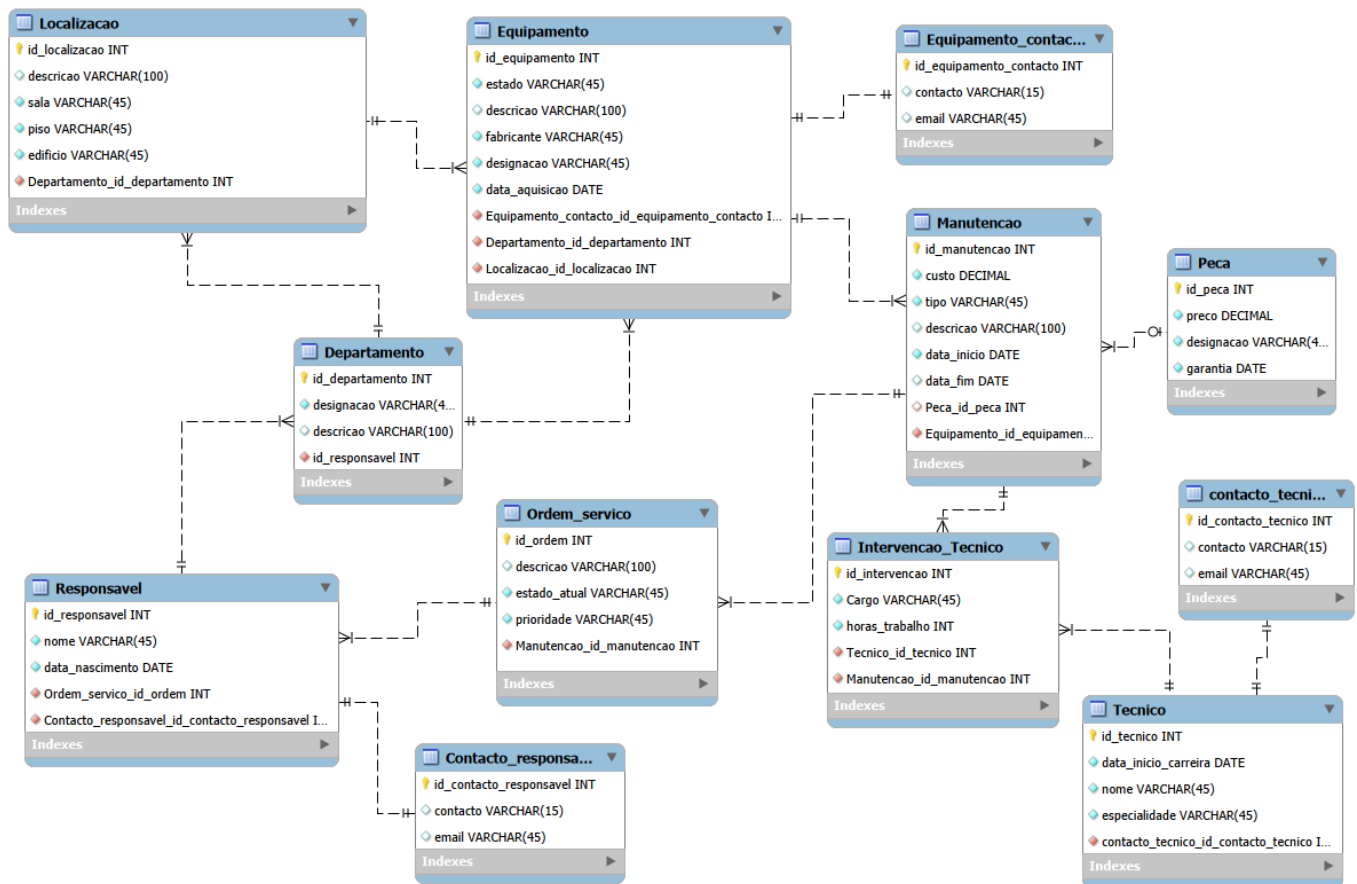


Figura 2: Modelo Lógico.

4 Modelo Físico

Um modelo físico de dados em SQL traduz de forma concreta e detalhada a estrutura de uma base de dados relacional. Nele são definidas as tabelas, as colunas e os respetivos tipos de dados, bem como as regras que garantem a integridade da informação como chaves primárias, chaves estrangeiras, índices e outras restrições.

Neste exemplo é criada a tabela *Departamento*, composta por quatro colunas: um identificador único gerado automaticamente (*id_departamento*), uma designação obrigatória, uma descrição opcional e uma referência ao responsável pelo departamento. O campo *id_departamento* é definido como chave primária, garantindo a unicidade de cada registo, enquanto *id_responsavel* funciona como chave estrangeira que estabelece a ligação com a tabela *Responsavel*, assegurando assim a integridade referencial entre as duas tabelas.

```

-----
-- Table `mydb`.`Departamento`
-----

DROP TABLE IF EXISTS `mydb`.`Departamento` ;

CREATE TABLE IF NOT EXISTS `mydb`.`Departamento` (
  `id_departamento` INT NOT NULL AUTO_INCREMENT,
  `designacao` VARCHAR(45) NOT NULL,
  `descricao` VARCHAR(100) NULL,
  `id_responsavel` INT NOT NULL,
  PRIMARY KEY (`id_departamento`),
  INDEX `fk_departamento_responsavel_idx` (`id_responsavel` ASC) VISIBLE,
  UNIQUE INDEX `idDepartamento_UNIQUE` (`id_departamento` ASC) VISIBLE,
  CONSTRAINT `fk_departamento_responsavel`
    FOREIGN KEY (`id_responsavel`)
    REFERENCES `mydb`.`Responsavel` (`id_responsavel`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;

```

Figura 3: Modelo físico - criação da tabela Departamento.

5 Documentação de Atributos

Na tabela apresentada no Anexo 2, é possível observar uma caracterização detalhada de cada atributo pertencente às diferentes entidades do modelo.

Destaca-se a coluna “Tipos de dados e tamanho”, que indica o tipo de dado associado a cada atributo, como número inteiro (INT), data (DATE), decimal (DECIMAL ou texto (VARCHAR), bem como o respetivo tamanho quando aplicável.

Para além disso, assumem especial importância as colunas de chave primária e chave estrangeira. A chave primária corresponde ao atributo (ou conjunto de atributos) que identifica de forma única cada registo numa tabela (por exemplo, *id_equipamento* na entidade *Equipamento* ou *id_manutenção* na entidade *Manutencao*).

Por outro lado, as chaves estrangeiras são responsáveis por estabelecer ligações entre tabelas distintas, referenciando chaves primárias de outras entidades (por exemplo, uma manutenção está associada a um equipamento através de uma chave estrangeira).

Estas chaves são fundamentais na organização de uma base de dados relacional, garantindo a integridade dos dados e facilitando a sua consulta e manipulação.

6 Povoamento

Após a criação das tabelas, é necessário proceder ao seu povoamento, ou seja, inserir dados na base de dados através do preenchimento das tabelas com registos contendo as informações relevantes. Para isso, utiliza-se a instrução SQL `INSERT INTO`. Na Figura 4, encontra-se exemplificada a inserção de dados nas tabelas *Tecnico* e *Responsavel*. Na tabela *Tecnico*, são registados cinco técnicos com os respetivos identificadores, datas de início de carreira, nomes e especialidades.

```
-- =====
-- 3. TABELA TECNICO
-- =====

INSERT INTO Tecnico (data_inicio_carreira, nome, especialidade, contacto_tecnico_id_contacto_tecnico) VALUES
('2015-01-15', 'Ricardo Pereira', 'Eletronica Medica', 1),
('2010-03-20', 'Marta Silveira', 'Imagiologia/Ressonancia', 2),
('2020-09-01', 'Joao Gouveia', 'Sistemas de Ventilacao', 3),
('2005-05-10', 'Antonio Costa', 'Instrumentacao Cirurgica', 4),
('2018-02-01', 'Beatriz Lopes', 'Monitorizacao Parametrica', 5);

-- =====
-- 4. TABELA RESPONSABEL
-- =====

INSERT INTO Responsavel (nome, data_nascimento, Ordem_servico_id_ordem, Contacto_responsavel_id_contacto_responsavel) VALUES
('Dra. Ana Martins', '1975-03-12', 1, 1),
('Dr. Carlos Santos', '1982-07-25', 2, 2),
('Eng. Ricardo Pereira', '1980-11-05', 3, 3),
('Dra. Sofia Oliveira', '1988-01-30', 4, 4),
('Dr. Miguel Ferreira', '1970-05-18', 5, 5);
```

Figura 4: Povoamento das entidades *Tecnico* e *Responsavel*.

7 Funções

As funções foram desenvolvidas com o objetivo de calcular e retornar atributos derivados que, não sendo armazenados fisicamente na base de dados, necessitam de ser computados de forma dinâmica sempre que solicitados. A sua implementação evita redundâncias de código, e garantindo que os valores derivados são sempre consistentes com o estado atual dos dados. Todas as funções foram declaradas como **DETERMINISTIC**, assegurando que para os mesmos argumentos de entrada o resultado retornado é sempre o mesmo. Nas secções seguintes, detalham-se as funções desenvolvidas e a lógica subjacente a cada uma.

7.1 Função *calcular_idade_equipamento*

Foi criada a função *calcular_idade_equipamento*, que permite determinar a idade de um equipamento (em anos) a partir da sua data de aquisição. A função recebe um argumento *p_idEquipamento* do tipo `INT`, correspondente ao

identificador do equipamento, e retorna um valor inteiro (**RETURNS INT**). Dentro do bloco **BEGIN...END**, é realizada uma consulta à tabela *Equipamento* para obter a *data_aquisicao* associada ao identificador fornecido, sendo de seguida utilizado **TIMESTAMPDIFF(YEAR, data_aquisicao, CURDATE())** para calcular a diferença em anos entre essa data e a data atual, valor que é retornado no final da função.

7.2 Função *calcular_duracao_manutencao*

Foi criada a função *calcular_duracao_manutencao*, que permite determinar a duração de uma manutenção em dias. A função recebe um argumento *p_id_manutencao* do tipo **INT**, correspondente ao identificador da manutenção, e retorna um valor inteiro. Dentro do bloco **BEGIN...END**, é realizada uma consulta à tabela *Manutencao* para obter as datas *data_inicio* e *data_fim* associadas ao identificador fornecido, sendo de seguida utilizado **DATEDIFF(data_fim, data_inicio)** para calcular a diferença em dias entre ambas, valor que é retornado no final da função.

7.3 Função *calcular_experiencia_tecnico*

Foi criada a função *calcular_experiencia_tecnico*, que permite determinar os anos de experiência profissional de um técnico a partir da sua data de início de carreira. A função recebe um argumento *p_id_tecnico* do tipo **INT**, correspondente ao identificador do técnico, e retorna um valor inteiro. Dentro do bloco **BEGIN...END**, é realizada uma consulta à tabela *Tecnico* para obter o campo *data_inicio_carreira* associado ao identificador fornecido, sendo de seguida utilizado **TIMESTAMPDIFF(YEAR, data_inicio_carreira, CURDATE())** para calcular a diferença em anos entre essa data e a data atual, valor que é retornado no final da função.

7.4 Função *equipamento_tem_manutencao_ativa*

Foi criada a função *equipamento_tem_manutencao_ativa*, que permite verificar se um determinado equipamento possui uma manutenção atualmente em curso. A função recebe um argumento *p_id_equipamento* do tipo **INT**, correspondente ao identificador do equipamento, e retorna um valor booleano (**RETURNS BOOLEAN**). Dentro do bloco **BEGIN...END**, é realizada uma consulta às tabelas *Manutencao* e *Ordem_servico*, contando o número de registos em que o equipamento indicado está associado a uma ordem de serviço cujo *estado_atual* não seja 'Concluida' nem 'Cancelada'. A função retorna **TRUE** caso esse total seja superior a zero, e **FALSE** caso contrário.

7.5 Função *obter_localizacao_equipamento*

Foi criada a função *obter_localizacao_equipamento*, que permite obter a localização completa de um equipamento numa única *string* formatada. A função recebe um argumento *p_id_equipamento* do tipo **INT**, correspondente ao identificador do equipamento, e retorna uma cadeia de caracteres (**RETURNS VARCHAR(150)**). Dentro do bloco **BEGIN...END**, é realizado um **JOIN** entre as tabelas *Equipamento* e *Localizacao*, sendo utilizado **CONCAT** para construir uma *string* com

o edifício, piso e sala associados ao equipamento, no formato 'Edifício: X — Piso: Y — Sala: Z', valor que é retornado no final da função.

7.6 Função *custo_total_manutencoes*

Foi criada a função *custo_total_manutencoes*, que permite calcular o custo acumulado de todas as manutenções registadas para um determinado equipamento. A função recebe um argumento *p_id_equipamento* do tipo INT, correspondente ao identificador do equipamento, e retorna um valor decimal (RETURNS DECIMAL(10,2)). Dentro do bloco BEGIN...END, é realizada uma consulta à tabela *Manutencao*, aplicando a função de agregação SUM sobre o campo custo para todos os registos associados ao equipamento indicado. É utilizado COALESCE para garantir que, na ausência de manutenções, a função retorna zero em vez de NULL, sendo esse valor retornado no final da função.

8 Procedures

Os procedimentos armazenados (*stored procedures*) foram desenhados para encapsular a lógica de negócio e automatizar as operações mais complexas e recorrentes do sistema. A sua implementação visa não só otimizar o desempenho das consultas, mas também assegurar a consistência e a integridade dos dados através de uma gestão rigorosa de transações e mecanismos de tratamento de erros (*exception handling*). Nas secções seguintes, detalham-se os procedimentos desenvolvidos e as respetivas regras de validação.

8.1 Procedimento *registar_avaria*

O procedimento *registar_avaria* permite abrir uma avaria no sistema. Recebe como parâmetros os identificadores do equipamento e da peça, o tipo de manutenção, o custo estimado, a descrição e a prioridade. O procedimento valida a existência do equipamento e da peça. De seguida, insere um registo na tabela '*Manutencao*' com a data de início atual e cria automaticamente uma ordem de serviço associada com o estado 'Pendente'.

8.2 Procedimento *concluir_manutencao*

O procedimento *concluir_manutencao* encerra uma manutenção em curso. Recebe o identificador da manutenção e verifica a sua existência e se já se encontra concluída. Utilizando uma transação, atualiza a data de fim da manutenção para a data atual.

8.3 Procedimento *adicionar_intervencao_tecnico*

O procedimento *adicionar_intervencao_tecnico* associa um técnico a uma intervenção, definindo o seu cargo e as horas de trabalho iniciais. Recebe os identificadores do técnico e da manutenção, garantindo que ambos existem e que as horas de trabalho são superiores a zero. Caso o parâmetro *p_mudar_ordem_execucao* seja ativado (1), o procedimento

atualiza o estado da ordem de serviço correspondente de 'Pendente' para 'Em Execução'. Toda a operação é protegida por uma transação.

8.4 Procedimento *abater_equipamento*

O procedimento *abater_equipamento* marca um equipamento como 'Inativo' e cancela as suas ordens de serviço pendentes. Após validar a existência do equipamento, utiliza uma transação para atualizar o estado na tabela *Equipamento* e alterar para 'Cancelada' o estado das ordens associadas que se encontrem 'Pendentes' ou 'Em Execução'.

8.5 Procedimento *alterar_prioridade_ordem*

O procedimento *alterar_prioridade_ordem* tem como objetivo atualizar a prioridade de uma ordem de serviço específica. O procedimento atualiza a tabela *Ordem_servico* com a nova prioridade e, caso a ordem não exista, levanta um erro para informar o utilizador.

8.6 Procedimento *listar_alertas*

O procedimento *listar_alertas* lista alertas operacionais cruciais para a gestão hospitalar. Recebe os limites de dias para manutenções abertas e ordens pendentes. Retorna três conjuntos de resultados: manutenções abertas há mais dias do que o limite definido, ordens de serviço pendentes há demasiado tempo, e equipamentos que não se encontram operacionais (estados 'Em Manutenção' ou 'Inativo').

8.7 Procedimento *validar_estado_ordem*

O procedimento *validar_estado_ordem* assegura que as transições de estado de uma ordem de serviço seguem um fluxo lógico (ex.: Pendente → Em Execução → Concluída ou Cancelada). Também uniformiza e normaliza as strings de estados para garantir consistência na base de dados, lidando com caracteres acentuados.

8.8 Procedimento *adicionar_responsavel*

O procedimento *adicionar_responsavel* regista um novo responsável na base de dados. Recebe o nome, data de nascimento, a ordem de serviço associada e os dados de contacto. Numa única transação, insere primeiramente os dados na tabela *Contacto_responsavel* e, de seguida, utiliza o identificador gerado para inserir o registo na tabela *Responsavel*.

8.9 Procedimento *adicionar_tecnico*

À semelhança do procedimento anterior, o *adicionar_tecnico* permite registar um novo técnico no sistema. Recebe as informações pessoais e de contacto do técnico. Insere os contactos na tabela *contacto_tecnico* e utiliza o identificador resultante para criar o técnico na tabela *Tecnico*, assegurando a integridade relacional.

8.10 Procedimento *adicionar_peca*

O procedimento *adicionar_peca* regista uma nova peça no catálogo. Valida se o preço inserido não é negativo e, em caso afirmativo, insere a nova peça na tabela *Peca*, devolvendo o identificador gerado.

8.11 Procedimento *abater_pecas*

O procedimento *abater_pecas* realiza uma limpeza ao inventário, apagando todas as peças cuja garantia já expirou (data de garantia inferior à data atual) e que nunca foram utilizadas em nenhuma manutenção registada no histórico.

9 *Triggers*

Os *triggers* foram desenvolvidos com o objetivo de automatizar comportamentos do sistema e garantir a integridade dos dados de forma transparente, sem depender de intervenção manual ou da camada aplicacional. A sua implementação assegura que as regras de negócio definidas são sempre aplicadas, independentemente da origem da operação que as despoleta. Os *triggers* desenvolvidos atuam em momentos distintos do ciclo de vida dos dados — antes ou após operações de inserção, atualização ou eliminação — cobrindo validações de domínio, controlo de coerência temporal e financeira, atualização automática de estados e proteção de registos críticos. Nas secções seguintes, detalham-se os *triggers* implementados e as respetivas regras de validação.

9.1 *Trigger trg_manutencao_validar_datas_insert*

Foi criado o trigger *trg_manutencao_validar_datas_insert*, que é ativado antes de cada inserção na tabela *Manutencao* (BEFORE INSERT). O *trigger* serve dois propósitos: em primeiro lugar, verifica se o equipamento indicado já possui uma manutenção em curso, contando os registos com *data_fim* a NULL para o mesmo *Equipamento_id_equipamento*. Caso exista pelo menos um, é lançado um erro impedindo a inserção; em segundo lugar, caso ambas as datas estejam preenchidas, valida que *data_inicio* não é posterior a *data_fim*, lançando igualmente um erro se essa condição for violada.

9.2 *Trigger trg_manutencao_validar_datas_update*

Este *trigger* é acionado antes de cada atualização na tabela *Manutencao* (BEFORE UPDATE). O seu funcionamento é semelhante à validação de datas do trigger anterior: sempre que ambos os campos *data_inicio* e *data_fim* estejam preenchidos no novo registo, verifica se *data_inicio* é posterior a *data_fim* e, caso essa condição se verifique, lança um erro, impedindo a atualização.

9.3 *Trigger trg_manutencao_estado_equipamento_insert*

Já este *trigger* é ativado após cada inserção na tabela *Manutencao* (AFTER INSERT). Verifica se o novo registo de manutenção não possui *data_fim* definida (o que indica que a manutenção foi iniciada mas ainda não foi concluída) e, nesse caso, atualiza automaticamente o campo estado do equipamento correspondente para 'Em Manutencao' na tabela *Equipamento*.

9.4 *Trigger trg_manutencao_estado_equipamento_update*

Por sua vez, o *trigger trg_manutencao_estado_equipamento_update* é ativado após cada atualização na tabela *Manutencao* (AFTER UPDATE). O *trigger* verifica se a manutenção transitou de um estado sem data de fim para um estado com *data_fim* definida, isto é, se foi concluída nessa atualização. Caso essa condição se verifique, são executadas duas ações automaticamente: o estado do equipamento associado é reposto para 'Operacional' na tabela *Equipamento*, e o *estado_atual* da ordem de serviço ligada a essa manutenção é atualizado para 'Concluída' na tabela *Ordem_servico*.

9.5 *Trigger trg_ordem_servico_validar_dominios_insert e trg_ordem_servico_validar_dominios_update*

Estes *triggers* atuam, respetivamente, antes de cada inserção e atualização na tabela *Ordem_servico* (BEFORE INSERT e BEFORE UPDATE). Ambos validam os valores dos campos prioridade e estado_atual, verificando se pertencem aos domínios definidos: para prioridade, os valores aceites são 'Baixa', 'Media', 'Alta' e 'Urgente'; para *estado_atual*, os valores aceites são 'Pendente', 'Em Curso', 'Concluída' e 'Cancelada'. Caso qualquer um dos valores não pertença ao respetivo domínio, é lançado um erro, impedindo a operação.

9.6 *Trigger trg_equipamento_proteger_delete e trg_equipamento_validar_estado_update*

Foram criados os *triggers trg_equipamento_validar_estado_insert* e *trg_equipamento_validar_estado_update*, ativados respetivamente antes de cada inserção e atualização na tabela *Equipamento* (BEFORE INSERT e BEFORE UPDATE). Ambos validam o valor do campo estado, verificando se pertence ao domínio definido: 'Operacional', 'Em Manutencao', 'Avariado' e 'Abatido'. Caso o valor indicado não pertença a esse conjunto, é lançado um erro, impedindo a operação.

9.7 *Trigger trg_equipamento_proteger_delete*

É ativado antes de cada eliminação na tabela *Equipamento* (BEFORE DELETE). O *trigger* realiza duas verificações sequenciais: primeiro, conta o número de manutenções ativas associadas ao equipamento, isto é, registos com *data_fim* a NULL e impede a eliminação caso esse valor seja superior a zero; de seguida, verifica se existem ordens de serviço associadas ao equipamento cujo *estado_atual* seja diferente de 'Concluída', impedindo igualmente a eliminação caso existam.

9.8 *Trigger trg_manutencao_validar_custo_insert*

Por último, o *trigger trg_manutencao_validar_custo_insert* é acionado antes de cada inserção na tabela *Manutencao* (BEFORE INSERT). O *trigger* apenas atua quando a manutenção a inserir tem uma peça associada (*Peca_id_peca* não é NULL). Nesse caso, consulta o campo *preco* da peça correspondente na tabela *Peca* e verifica se o custo da manutenção é inferior a esse valor.

10 *Queries*

As *queries* foram desenvolvidas com o objetivo de responder a necessidades concretas de consulta e análise operacional que não se encontram diretamente cobertas pelas *views* existentes, tirando partido de subconsultas correlacionadas, funções de agregação e expressões condicionais para produzir resultados mais específicos e detalhados. Ao contrário das *views*, que representam consultas fixas e reutilizáveis, as *queries* aqui apresentadas foram concebidas para cenários de interrogação pontuais, respondendo a questões de gestão como a disponibilidade de técnicos, o controlo de garantias, a identificação de equipamentos em risco e a análise financeira das intervenções. Nas secções seguintes, detalham-se as *queries* desenvolvidas e os objetivos que cada uma visa cumprir.

10.1 *Query 1 - Técnicos Disponíveis por Especialidade*

A primeira *query* tem como objetivo identificar os técnicos que se encontram disponíveis, ou seja, que não estão atualmente alocados a nenhuma manutenção em curso. Para cada técnico disponível, são retornados o identificador, o nome, a especialidade e os anos de experiência, calculados através da função *calcular_experiencia_tecnico*. A condição de disponibilidade é verificada através de uma subconsulta com NOT EXISTS, que percorre as tabelas *Intervencao_Tecnico* e *Manutencao* à procura de intervenções associadas ao técnico em que o campo *data_fim* seja NULL. Os resultados são ordenados por especialidade e, dentro de cada especialidade, por anos de experiência de forma decrescente.

10.2 *Query 2 - Peças com Garantia a Expirar*

Esta *query* identifica as peças cujo prazo de garantia expira nos próximos 365 dias. Para cada peça dentro desse intervalo, são retornados o identificador, a designação, o preço, a data de garantia, o número de dias restantes até à expiração e o número de vezes que a peça foi utilizada em manutenções. O filtro temporal é aplicado através da cláusula WHERE com a condição BETWEEN CURDATE() e DATE_ADD(CURDATE(), INTERVAL 365 DAY), e o número de utilizações é obtido através de um LEFT JOIN com a tabela *Manutencao* combinado com a função de agregação COUNT. Os resultados são ordenados pelo número de dias restantes de forma crescente, apresentando primeiro as peças com garantia a expirar mais brevemente.

10.3 *Query 3 - Equipamentos sem Histórico de Manutenção*

Permite identificar os equipamentos que nunca tiveram qualquer manutenção registada no sistema, o que representa um potencial risco operacional, especialmente em equipamentos de maior antiguidade. São retornados o identificador, a designação, o fabricante, o estado, a data de aquisição, a idade em anos calculada pela função *calcular_idade_equipamento* e o departamento associado. A ausência de manutenções é verificada através de uma subconsulta com `NOT EXISTS` sobre a tabela *Manutencao*. Os resultados são ordenados pela data de aquisição de forma crescente, colocando em primeiro lugar os equipamentos mais antigos sem histórico de manutenção.

10.4 *Query 4 - Ranking de Técnicos por Custo Gerido*

Esta *query* constrói um *ranking* dos técnicos ordenado pelo custo total das manutenções em que intervieram. Para cada técnico, são calculados o número de manutenções distintas em que participou, o total de horas de trabalho registadas, o custo total das manutenções geridas e o custo médio por hora, obtido através do rácio entre o custo total e as horas totais. Os dados são obtidos através de junções entre as tabelas *Tecnico*, *Intervencao_Tecnico* e *Manutencao*, utilizando funções de agregação como `COUNT`, `SUM` e `ROUND`. Os resultados são ordenados pelo custo total gerido de forma decrescente.

10.5 *Query 5 - Histórico Completo de um Equipamento*

Já esta *query* permite visualizar o histórico completo de manutenções de um equipamento específico, neste caso o equipamento com identificador igual a 1. Para cada manutenção registada são retornados a designação e o fabricante do equipamento, o tipo de manutenção, as datas de início e fim, a duração em dias calculada com `DATEDIFF` e `COALESCE` para tratar manutenções ainda em curso, o custo, a peça utilizada e o seu preço, o técnico interveniente com o respetivo cargo e horas de trabalho, e o estado atual da ordem de serviço associada. A consulta recorre a múltiplos `LEFT JOIN` para garantir que são incluídas manutenções mesmo que não tenham peça, técnico ou ordem de serviço associados. Os resultados são ordenados pela data de início de forma decrescente.

10.6 *Query 6 - Estado Operacional dos Equipamentos por Departamento*

Apresenta, por departamento, a distribuição dos equipamentos segundo o seu estado operacional. Para cada departamento são retornados o nome do responsável e a contagem de equipamentos em cada estado: Operacional, Em Manutencao, Avariado e Abatido, bem como o total de equipamentos e a percentagem de equipamentos não operacionais. Esta percentagem é calculada através do rácio entre o número de equipamentos com estado diferente de 'Operacional' e o total de equipamentos do departamento, multiplicado por 100. A contagem por estado é obtida com expressões `CASE WHEN` dentro da função `SUM`. Os resultados são ordenados pela percentagem de equipamentos não operacionais de forma decrescente, identificando os departamentos com maior impacto operacional.

10.7 *Query 7 - Manutenções com Maior Custo Total*

Por último, esta *query* identifica as cinco manutenções com maior custo total, entendido como a soma do custo base da manutenção com o preço da peça utilizada, recorrendo a **COALESCE** para atribuir o valor zero quando não existe peça associada. Para cada manutenção são retornados o identificador, o tipo, a descrição, as datas de início e fim, o equipamento e respetivo fabricante, o departamento, o custo da manutenção, o custo da peça, o custo total, a prioridade e o estado atual da ordem de serviço. Os dados são obtidos através de junções entre as tabelas *Manutencao*, *Equipamento*, *Departamento*, *Peca* e *Ordem_servico*, utilizando **LEFT JOIN** nas tabelas opcionais. Os resultados são ordenados pelo custo total de forma decrescente e limitados aos cinco primeiros registos através da cláusula **LIMIT 5**.

11 *Views*

As *views* foram desenvolvidas com o objetivo de simplificar o acesso à informação agregada e facilitar a reutilização de consultas complexas ao longo do sistema. Permitem que a informação seja consultada de forma mais intuitiva e sem necessidade de reproduzir lógica repetitiva em cada interrogação. A sua implementação contribui igualmente para a separação entre a estrutura interna da base de dados e a forma como a informação é apresentada e consumida. Nas secções seguintes, detalham-se as *views* desenvolvidas e os requisitos funcionais que cada uma visa satisfazer.

11.1 *View vw_custo_total_equipamento*

A *view vw_custo_total_equipamento* tem como objetivo apresentar o custo total real associado a cada equipamento, calculado como a soma do custo base das manutenções registadas com o preço das peças utilizadas nessas intervenções. Para cada equipamento são retornados a designação, o fabricante, o estado, o departamento, o número total de manutenções, o custo base total, o custo total das peças e o custo total real. Os dados são obtidos através de junções entre as tabelas *Equipamento* e *Departamento*, e de **LEFT JOIN** com as tabelas *Manutencao* e *Peca*, garantindo que equipamentos sem manutenções registadas são igualmente incluídos. A função **COALESCE** é utilizada para atribuir o valor zero ao custo das peças quando não existe peça associada, evitando resultados nulos. Os valores monetários são arredondados a duas casas decimais com **ROUND**, e os resultados são ordenados pelo custo total real de forma decrescente.

11.2 *View vw_garantia_pecas*

Tem como objetivo apresentar o estado de garantia de cada peça utilizada em manutenções, permitindo monitorizar quais as peças que se encontram dentro do prazo de garantia e quais já o ultrapassaram. Para cada registo são retornados o identificador e as datas da manutenção, o equipamento e respetivo fabricante, o departamento, e os detalhes da peça utilizada, nomeadamente a designação, o preço e a data de fim de garantia. O estado da garantia é determinado através de uma expressão **CASE WHEN** que compara a data de garantia com a data atual, retornando 'Em garantia' ou 'Fora de garantia', e o número de dias restantes é calculado com **DATEDIFF**. Os resultados são ordenados pela data de garantia de forma crescente, apresentando primeiro as peças com garantia a expirar mais brevemente.

11.3 *View vw_rastreabilidade_intervencoes*

Esta *view* tem como objetivo fornecer um registo completo e detalhado de todas as intervenções técnicas realizadas, permitindo rastrear quem interveio em cada manutenção, em que equipamento, em que localização e com que cargo. Para cada intervenção são retornados os dados do equipamento, o departamento, a localização física (edifício, piso e sala), os detalhes da manutenção, o técnico responsável com a sua especialidade e cargo, as horas de trabalho despendidas e os anos de experiência calculados pela função *calcular_experiencia_tecnico*. Os dados são obtidos através de junções entre as tabelas *Intervencao_Tecnico*, *Tecnico*, *Manutencao*, *Equipamento*, *Departamento* e *Localizacao*. Os resultados são ordenados por equipamento e, dentro de cada equipamento, pela data de início da manutenção de forma decrescente.

11.4 *View vw_downtime_equipamentos*

Permite quantificar o tempo de paragem acumulado de cada equipamento devido a manutenções, fornecendo indicadores relevantes para a gestão da disponibilidade operacional da frota. Para cada equipamento são retornados a designação, o fabricante, o estado, o departamento, a idade em anos calculada pela função *calcular_idade_equipamento*, o número total de manutenções, os dias de paragem totais, a média de dias por manutenção e as datas de início e fim da última paragem registada. A duração de cada manutenção é calculada com *DATEDIFF*, utilizando *COALESCE* para substituir a data de fim por *CURDATE()* nos casos em que a manutenção ainda se encontra em curso. Os resultados são ordenados pelo total de dias de paragem de forma decrescente.

11.5 *View vw_localizacao_equipamentos*

Apresenta a localização física atual de todos os equipamentos registados no sistema. Para cada equipamento são retornados a designação, o fabricante, o estado, o departamento, o nome do responsável pelo departamento, os detalhes da localização (edifício, piso, sala e descrição) e uma *string* formatada com a localização completa, obtida através da função *obter_localizacao_equipamento*. Os dados são obtidos através de junções entre as tabelas *Equipamento*, *Departamento*, *Localizacao* e *Responsavel*. Os resultados são ordenados por departamento e, dentro de cada departamento, por designação do equipamento.

11.6 *View vw_ordens_abertas*

Esta *view* lista todas as ordens de serviço que se encontram atualmente em aberto, ou seja, com estado diferente de 'Concluída' ou 'Cancelada', permitindo monitorizar as intervenções pendentes e em curso. Para cada ordem são retornados o identificador, a prioridade, o estado atual, a descrição, o tipo de manutenção associada, a data de início, o número de dias em aberto calculado com *DATEDIFF*, o equipamento e o seu estado, o departamento e a localização física. Os resultados são ordenados por prioridade através da função *FIELD*, apresentando primeiro as ordens de prioridade 'Alta', seguidas das de prioridade 'Media' e por fim as de prioridade 'Baixa'.

11.7 View *vw_tecnico_resumo*

A *view vw_tecnico_resumo* apresenta um resumo agregado da atividade de cada técnico registado no sistema. Para cada técnico são retornados o identificador, o nome, a especialidade, os anos de experiência calculados pela função *calcular_experiencia_tecnico*, o número total de intervenções realizadas, o total de horas de trabalho acumuladas e a média de horas por intervenção. Os dados são obtidos através de um **LEFT JOIN** entre as tabelas *Tecnico* e *Intervencao.Tecnico*, garantindo que técnicos sem intervenções registadas são igualmente incluídos. Os valores de horas são arredondados a uma casa decimal com **ROUND**, e os resultados são ordenados pelo total de horas de forma decrescente.

11.8 View *vw_custos_por_departamento*

Por último, a *view vw_custos_por_departamento* tem como objetivo apresentar uma análise financeira agregada por departamento, consolidando os custos de todas as manutenções e peças associadas aos equipamentos de cada departamento. Para cada departamento são retornados o nome do responsável, o número total de equipamentos, o número total de manutenções, o custo base total, o custo total das peças, o custo total combinado e o custo médio por manutenção. Os dados são obtidos através de junções entre as tabelas *Departamento*, *Responsavel* e *Equipamento*, e de **LEFT JOIN** com as tabelas *Manutencao* e *Peca*, garantindo que departamentos sem manutenções registadas são igualmente incluídos. A função **COALESCE** é utilizada para tratar valores nulos no custo das peças, e os valores monetários são arredondados a duas casas decimais com **ROUND**. Os resultados são ordenados pelo custo total de forma decrescente.

12 Conclusão

Em síntese, o presente trabalho focou-se na conceção e desenvolvimento de uma base de dados relacional para suporte à gestão de manutenção de equipamentos, recorrendo à plataforma MySQL Workbench como ferramenta principal de modelação e implementação. O sistema concebido revelou-se uma resposta adequada às necessidades identificadas, possibilitando o registo, acompanhamento e consulta de informação relativa a equipamentos, departamentos, técnicos, ordens de serviço e intervenções de manutenção, de forma estruturada e eficiente. A definição de regras de integridade referencial, restrições de domínio e funções auxiliares contribuiu para garantir a consistência e fiabilidade dos dados armazenados. No decorrer do projeto, tornou-se evidente a relevância de uma modelação cuidada e faseada, desde o levantamento de requisitos até à implementação física, uma vez que decisões tomadas nas fases iniciais têm impacto direto na qualidade e robustez da solução final.

No decorrer do projeto, recorreu-se também ao apoio de ferramentas de Inteligência Artificial generativa em algumas fases do desenvolvimento. Este auxílio revelou-se particularmente útil em tarefas de maior volume, nomeadamente no povoamento da base de dados com registos realistas e coerentes, na elaboração de algumas *queries* de consulta mais elaboradas e na estruturação inicial de certas *procedures*. O recurso a estas ferramentas não substituiu o raciocínio crítico nem as decisões de modelação, que foram sempre da responsabilidade da equipa, mas permitiu acelerar fases mais repetitivas do trabalho e explorar abordagens alternativas de forma mais ágil.

Em conclusão, este projeto permitiu consolidar e aplicar em contexto prático os conhecimentos adquiridos ao longo da unidade curricular, evidenciando de forma concreta a utilidade das bases de dados relacionais na automatização e organização de processos reais no seio de uma organização.

A Anexo 1: Requisitos da base de dados

Nº	Tipo	Descrição
1	Descrição	A base de dados deve conter 12 tabelas normalizadas que representem todas as entidades do domínio: Equipamento, Manutenção, Ordem de Serviço, Técnico, Peça, Localização, Departamento, Responsável e respetivas tabelas de contacto.
2	Descrição	Cada tabela deve ter uma chave primária numérica auto-incremental (AUTO_INCREMENT), garantindo unicidade de cada registo.
3	Descrição	As relações entre entidades devem ser garantidas por chaves estrangeiras (FOREIGN KEY), com ON DELETE NO ACTION e ON UPDATE NO ACTION, prevenindo eliminações acidentais de dados relacionados.
4	Descrição	As colunas utilizadas em operações de JOIN frequentes devem estar indexadas, garantindo desempenho aceitável nas consultas.
5	Descrição	Os atributos derivados (idade do equipamento, duração da manutenção, experiência do técnico, localização completa, custo total e verificação de manutenção ativa) devem ser calculados por funções e não armazenados fisicamente, evitando redundância e inconsistências.
6	Descrição	Os campos obrigatórios de cada tabela devem ser definidos com restrição NOT NULL, garantindo que não é possível inserir registos incompletos em atributos essenciais como nome, estado, datas e custo.
7	Descrição	Devem existir views que encapsulem as consultas complexas mais recorrentes, funcionando como camada de acesso a informação agregada (custos, garantias, rastreabilidade, downtime, localização, ordens abertas, resumo de técnicos e custos por departamento).
8	Manipulação	A base de dados deve ser populada com dados realistas, respeitando a ordem de inserção imposta pelas chaves estrangeiras: tabelas de contacto → técnicos/responsáveis → departamentos → localizações → equipamentos → manutenções → ordens de serviço → intervenções.
9	Manipulação	O sistema deve suportar o registo de uma nova avaria/manutenção através de uma stored procedure que cria automaticamente a manutenção e a ordem de serviço associada.
10	Manipulação	O sistema deve permitir concluir uma manutenção em curso, atualizando a data de fim numa única operação.
11	Manipulação	O sistema deve permitir associar um técnico a uma manutenção, registando o cargo e as horas de trabalho, com opção de atualizar o estado da ordem de serviço para 'Em Execução'.
12	Manipulação	O sistema deve permitir abater um equipamento, cancelando automaticamente todas as ordens de serviço pendentes ou em execução associadas.
13	Manipulação	O sistema deve permitir criar técnicos e responsáveis juntamente com os respetivos contactos numa única operação transacional.
14	Manipulação	O sistema deve emitir alertas operacionais para manutenções em aberto há demasiado tempo, ordens pendentes antigas e equipamentos não operacionais, com limites de dias configuráveis.
15	Manipulação	O sistema deve disponibilizar queries de apoio à gestão, organizadas por requisito funcional: custos, garantias, rastreabilidade, downtime, ordens de serviço, localização e verificação de estado.
16	Controlo	O sistema deve impedir a inserção de valores inválidos nos campos de estado do equipamento (Operacional, Em Manutencao, Avariado, Abatido) e de prioridade e estado da ordem de serviço (Pendente, Em Curso, Concluida, Cancelada), através de triggers de validação de domínio.
17	Controlo	O sistema deve impedir a abertura de uma nova manutenção para um equipamento que já tenha uma manutenção em curso, garantindo que não existem manutenções ativas duplicadas para o mesmo equipamento.
18	Controlo	O sistema deve atualizar automaticamente o estado do equipamento para 'Em Manutencao' ao registar uma nova manutenção sem data de fim, e repô-lo como 'Operacional' ao concluí-la.

Figura 5: Tabela de documentação de requisitos.

Nº	Tipo	Descrição
19	Controlo	O sistema deve validar a coerência temporal das manutenções, garantindo que a data de fim, quando definida, é sempre posterior à data de início.
20	Controlo	O sistema deve fechar automaticamente a ordem de serviço associada quando a manutenção correspondente é concluída (definição de data de fim).
21	Controlo	O sistema deve impedir a eliminação de equipamentos que possuam manutenções ativas ou ordens de serviço não concluídas.
22	Controlo	O sistema deve garantir que o custo registado numa manutenção não é inferior ao preço da peça utilizada, assegurando coerência financeira.
23	Controlo	As operações que envolvam múltiplas tabelas devem ser executadas de forma atómica, com reversão automática em caso de erro, garantindo que a base de dados nunca fica em estado inconsistente.
24	Controlo	As <i>stored procedures</i> devem validar a existência das entidades referenciadas (equipamento, peça, técnico, manutenção, ordem de serviço) antes de executar qualquer operação, retornando mensagens de erro explícitas em caso de dados inválidos.

Figura 6: Tabela de documentação de requisitos (continuação).

B Anexo 2: Tabela de Atributos

Nome da Entidade	Atributo	Descrição do atributo	Tipo de dados e tamanho	Nulo	Multi-valor	Chave primária	Chave estrangeira	Atributo composto
Equipamento	id_equipamento	Identificador do equipamento	INT	Não	Não	Sim	Não	Não
	estado	Estado atual do equipamento	VARCHAR(45)	Não	Não	Não	Não	Não
	descricao	Descrição do equipamento	VARCHAR(100)	Não	Não	Não	Não	Não
	fabricante	Fabricante do equipamento	VARCHAR(45)	Sim	Não	Não	Não	Não
	designacao	Designação do equipamento	VARCHAR(45)	Não	Não	Não	Não	Não
	data_aquisicao	Data de aquisição do equipamento	DATE	Sim	Não	Não	Não	Não
	Equipamento_contacto_id	Identificador do contacto do equipamento	INT	Não	Não	Não	Sim	Não
	Departamento_id_departamento	Identificador do departamento	INT	Não	Não	Não	Sim	Não
[Sub-tabela] Equipamento_contacto	id_equipamento_contacto	Identificador do contacto do equipamento	INT	Não	Não	Sim	Não	Não
	contacto_email	Contacto do equipamento Email de contacto do equipamento	VARCHAR(15) VARCHAR(45)	Não Sim	Sim Sim	Não Não	Não Não	Não Não
Departamento	id_departamento	Identificador do departamento	INT	Não	Não	Sim	Não	Não
	designacao	Designação do departamento	VARCHAR(45)	Não	Não	Não	Não	Não
	descricao	Descrição do departamento	VARCHAR(100)	Sim	Não	Não	Não	Não
	id_responsavel	Identificador do responsável	INT	Não	Não	Não	Sim	Não
Localizacao	id_localizacao	Identificador da localização	INT	Não	Não	Sim	Não	Não
	descricao	Descrição da localização	VARCHAR(100)	Sim	Não	Não	Não	Não
	sala	Sala da localização	VARCHAR(45)	Sim	Não	Não	Não	Sim
	piso	Piso da localização	VARCHAR(45)	Sim	Não	Não	Não	Sim
	edificio	Edifício da localização	VARCHAR(45)	Sim	Não	Não	Não	Sim
	Departamento_id_departamento	Identificador do departamento	INT	Não	Não	Não	Sim	Não
Responsavel	id_responsavel	Identificador do responsável	INT	Não	Não	Sim	Não	Não
	nome	Nome do responsável	VARCHAR(45)	Não	Não	Não	Não	Não
	data_nascimento	Data de nascimento do responsável	DATE	Sim	Não	Não	Não	Não
	Ordem_de_servico_id_ordem	Identificador da ordem de serviço	INT	Não	Não	Não	Sim	Não
[Sub-tabela] Contacto_responsavel	Contacto_responsavel_id	Identificador do contacto do responsável	INT	Não	Não	Não	Sim	Não
	id_contacto_responsavel	Identificador do contacto do responsável	INT	Não	Não	Sim	Não	Não
	contacto_email	Contacto do responsável Email do responsável	VARCHAR(15) VARCHAR(45)	Não Sim	Sim Sim	Não Não	Não Não	Não Não
Ordem_de_servico	id_ordem	Identificador da ordem de serviço	INT	Não	Não	Sim	Não	Não
	descricao	Descrição da ordem de serviço	VARCHAR(100)	Não	Não	Não	Não	Não
	estado_atual	Estado atual da ordem de serviço	VARCHAR(45)	Não	Não	Não	Não	Não
	prioridade	Nível de prioridade da ordem de serviço	VARCHAR(45)	Não	Não	Não	Não	Não
	Manutencao_id_manutencao	Identificador da manutenção	INT	Não	Não	Não	Sim	Não
Manutencao	id_manutencao	Identificador da manutenção	INT	Não	Não	Sim	Não	Não
	custo	Custo da manutenção	DECIMAL	Sim	Não	Não	Não	Não
	tipo	Tipo de manutenção	VARCHAR(45)	Não	Não	Não	Não	Não
	descricao	Descrição da manutenção	VARCHAR(100)	Sim	Não	Não	Não	Não
	data_inicio	Data de início da manutenção	DATE	Não	Não	Não	Não	Não
	data_fim	Data de fim da manutenção	DATE	Sim	Não	Não	Não	Não
	Peca_id_pecas	Identificador da peça	INT	Sim	Não	Não	Sim	Não
Peca	Equipamento_id_equipamento	Identificador do equipamento	INT	Não	Não	Não	Sim	Não
	id_pecas	Identificador da peça	INT	Não	Não	Sim	Não	Não
	preco	Preço da peça	DECIMAL	Não	Não	Não	Não	Não
	designacao	Designação da peça	VARCHAR(45)	Não	Não	Não	Não	Não
Intervencao_tecnico	garantia	Data de garantia da peça	DATE	Sim	Não	Não	Não	Não
	id_intervencao	Identificador da intervenção	INT	Não	Não	Sim	Não	Não
	Cargo	Cargo do técnico na intervenção	VARCHAR(45)	Sim	Não	Não	Não	Não
	horas_trabalho	Horas de trabalho na intervenção	INT	Não	Não	Não	Não	Não
Tecnico	Tecnico_id_tecnico	Identificador do técnico	INT	Não	Não	Não	Sim	Não
	Manutencao_id_manutencao	Identificador da manutenção	INT	Não	Não	Não	Sim	Não
	id_tecnico	Identificador do técnico	INT	Não	Não	Sim	Não	Não
	data_inicio_carreira	Data de início de carreira do técnico	DATE	Sim	Não	Não	Não	Não
[Sub-tabela] Contacto_tecnico	nome	Nome do técnico	VARCHAR(45)	Não	Não	Não	Não	Não
	especialidade	Especialidade do técnico	VARCHAR(45)	Sim	Não	Não	Não	Não
	contacto_tecnico_id	Identificador do contacto do técnico	INT	Não	Não	Não	Sim	Não
	id_contacto_tecnico	Identificador do contacto do técnico	INT	Não	Não	Sim	Não	Não
[Sub-tabela] Contacto_tecnico	contacto_email	Contacto do técnico Email do técnico	VARCHAR(15) VARCHAR(45)	Não Sim	Sim Sim	Não Não	Não Não	Não Não

Figura 7: Tabela de documentação de atributos.